

DeepRazor: Object Removal Web Application

Project Summary

This project implements an object removal model using the **Contextual Residual Aggregation (CRA)** approach [2], leveraging the **RORD dataset** [1]. It features a two-stage generator (coarse and refine) alongside a discriminator for adversarial training.

Project Structure

InpaintingModule

- **Logs/** — CSV logs for training/validation metrics.
- **Results/** — Stores output images and comparison plots.
- **commands.txt** — Saved terminal commands.
- **env_var.py** — Configuration settings.
- **quick.py** — Shortcut script for testing.

src/ – Core Implementation

- **dataset/** — DataLoader for training/validation.
- **model/** — Core components (generator, discriminator, etc.).
- **utils/** — Utility functions (logging, training, validation, etc.).

Entry Scripts

- **train.py** — Model training.
- **test.py** — Tests the model.
- **validate.py** — Validation metrics.
- **util.py** — Common utilities.

Deployment – Streamlit App

- **app.py** — Streamlit app UI and state management.
- **draw_mask.py** — Interactive masking tool.
- **inpaint anything.py** — Core inpainting logic.

- `segment_anything.py` — Integrates segmentation (likely SAM).
- `utils.py` — Session management and preprocessing utilities.

`onnx_gen_models/` – ONNX Model Export

- `export_to_onnx.py` — Converts models to ONNX format.
- `run_onnx.py` — Inference with ONNX runtime.
- `ia_gen.onnx*` — Exported inpainting models.

This version summarizes the core details, architecture, and key components of your project without losing important context. Let me know if you need further refinements!

Dataset Preparation

- **Dataset:** Uses the **RORD dataset** which provides paired images (with objects and corresponding object-free backgrounds) and dense binary masks.
- **Sampling & Batching:**
 - Approximately **24K training** and **4K validation** instances are selected.
 - Each instance is a triplet:
 - **Image:** Contains the target object.
 - **Mask:** A binary mask indicating object regions.
 - **Ground Truth:** The object-free (background) image.
 - Training is performed in batches of **16 triplets**.

Generator Architecture

The generator is defined in `generator.py` and follows a two-stage, coarse-to-fine design inspired by CRA [2].

1. Coarse Generator (class `Coarse` in `coarse.py`)

- **Input:**
 - The original image and mask are resized to 256×256.
 - A masked image is computed as:

$$\text{masked_img} = \text{img} * (1 - \text{mask}) + \text{mask}$$
 - The masked image is concatenated with the mask to form a 4-channel tensor.
- **Architecture:**

- **Encoder:**
 - Uses a sequence of **GatedConv2d** layers.
 - **Single-channel gated convolutions** are applied (via the `Conv_SC` module) to compute a soft gating mask.
- **Residual Blocks:**
 - Blocks `coarse2` to `coarse8` use dilated convolutions with dilation rates 1, 2, 4, 8, and 16.
 - Skip connections add outputs from earlier layers to later ones.
- **Decoder:**
 - Uses **TransposeGatedConv2d** layers to upsample features.
 - The final layer is a gated convolution with activation set to `'none'`, followed by a `Tanh` activation, and the output is clamped between 0 and 1.
- **Output:**
 - Produces a coarse inpainted image which is then upsampled (e.g., to 512×512) and blended with the original image to fill the masked regions.

2. Refinement Generator (class `Refine` in `refine.py`)

- **Input:**
 - The coarse output is merged with the original image (holes replaced with coarse prediction) and is passed along with the mask.
- **Architecture:**
 - **Encoder:**
 - Composed of downsampling blocks (`ref_block_1` , `ref_block_2` , `ref_block_3`) using **GatedConv2d** layers.
 - **Feature Refinement:**
 - Uses dilated blocks (`dilated_block_1` , `dilated_block_2` , `dilated_block_3`) to further process features.
 - **Depth-separable gated convolutions** (via the `Conv_DS` module) are employed in this stage for efficient, channel-wise filtering.
 - **Contextual Attention Module (CAM):**
 - Implemented in `cam.py`, it computes patch-based cosine similarity to transfer contextual information from unmasked regions to occluded areas.
 - **Optional Transformer Bottleneck:**
 - A Swin Transformer block (from `swin_transformer.py`) is optionally applied during the last 10 epochs to incorporate

- global context.
- **Decoder:**
 - Upsampling is done using transposed blocks (`transposed_ref_block_3` , `transposed_ref_block_2` , `transposed_ref_block_1`).
 - Skip connections integrate low-level features extracted via the `l1f_p1` layers.
- **Output:**
 - Yields the final refined inpainted image, clamped between 0 and 1.

Discriminator (class `Discriminator` in `discriminator.py`)

- **Input:**
 - Constructs a 4-channel tensor by multiplying the image with the mask (to emphasize masked regions) and concatenating the mask.
- **Architecture:**
 - Consists of a series of **Conv2dLayer** blocks.
 - Uses instance normalization (`norm = 'in'`) and LeakyReLU activation (`activation = 'lrelu'`).
 - **Spectral normalization** (`sn = True`) is applied from the second layer onward to stabilize adversarial training.
- **Output:**
 - Produces a single-channel authenticity map that distinguishes between real and generated images.

Key Innovations

- **Gated Convolutions:**
 - **Coarse Network:** Uses single-channel gated convolutions (via `Conv_SC`) to generate a soft gating mask that is broadcast across feature channels.
 - **Refine Network:** Uses depth-separable gated convolutions (via `Conv_DS`) to reduce computational cost while preserving spatial information.
- **Context Aggregation:**
 - Multi-dilated residual blocks aggregate context over various scales (dilations 1–16) following the CRA mechanism [2].

- **Contextual Attention:**
 - The CAM module computes patch-based cosine similarities to transfer context from unmasked to masked regions, enhancing inpainting detail.
- **Transformer Integration:**
 - An optional Swin Transformer bottleneck in the refine stage (introduced in the last 10 epochs) improves global context modeling.
- **Adversarial Training:**
 - The Discriminator, with spectral and instance normalization, helps ensure realistic and coherent inpainting results.

References

1. Sagong, M.-C., Yeo, Y.-J., Jung, S.-W., & Ko, S.-J. (2022). *RORD: A Real-world Object Removal Dataset*. Proceedings of the British Machine Vision Conference (BMVC).
2. Yi, Z., Tang, Q., Azizi, S., Jang, D., & Xu, Z. (2020). *Contextual Residual Aggregation for Ultra High-Resolution Image Inpainting*. arXiv preprint [arXiv:2005.09704](https://arxiv.org/abs/2005.09704).
3. Yu, J., Lin, Z., Yang, J., Shen, X., Lu, X., & Huang, T. S. (2019). *Free-form Image Inpainting with Gated Convolution*. Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 4471–4480.
4. DeepFillv2 Gated Convolution Code. [DeepFillv2 Code](#)